

Nonparametric Classification and Regression with k-Nearest Neighbors, Edited KNN, and Condensed KNN

Isaac Wilkey

*Department of Computer Science
Montana State University
Bozeman, MT, USA*

ISAAC.WILKEY@STUDENT.MONTANA.EDU

Riley Smith

*Department of Computer Science
Montana State University
Bozeman, MT, USA*

RILEY.SMITH13@STUDENT.MONTANA.EDU

Editor: Isaac & Riley

Abstract

This project implements and evaluates nonparametric machine learning algorithms for both classification and regression tasks across six diverse datasets from the UCI Machine Learning Repository. We implement a baseline Null Model, a standard k-Nearest Neighbor (KNN) approach, and two data reduction techniques: Edited KNN and Condensed KNN. For regression, we utilize a Gaussian (Radial Basis Function) kernel to perform weighted averaging of neighbors. Our experimental process employs 5×2 cross-validation to statistically compare the performance of the standard KNN against the reduced dataset variants. We evaluate the impact of hyperparameter tuning for k , the kernel bandwidth γ , and the regression error threshold ϵ . Results demonstrate the trade-offs between training set size and predictive accuracy. This shows how the data reduction can maintain performance while increasing computational efficiency.

Keywords: nonparametric learning, k-nearest neighbors, data reduction, Edited KNN, Condensed KNN, Gaussian kernel

1 Problem Statement and Hypothesis

The goal of this project is to implement and analyze nonparametric methods for classification and regression. Unlike parametric models, nonparametric methods do not assume a fixed functional form for the underlying mapping. This makes them flexible for complex datasets. However, they often suffer from high computational costs during inference as the training set grows.

We hypothesize that:

1. The tuned KNN will have significantly better performance than the Null Model across all six datasets, proving that local feature proximity is a strong predictor for these tasks.
2. Edited KNN will provide a "cleaner" decision boundary by removing noisy examples, potentially improving accuracy or maintaining it with a much smaller dataset.

3. Condensed KNN will achieve the highest rate of data reduction while maintaining acceptable performance, as it only retains the most critical examples used to define the class boundaries.
4. For regression, the tuning of the bandwidth parameter γ will be more critical to the Mean Squared Error (MSE) than the choice of k alone.

2 Experimental Approach and Project Design

Our experimental pipeline consisted of data acquisition, preprocessing, model tuning, and statistical evaluation.

2.1 Datasets and Preprocessing

We utilize six datasets from the UCI repository. To ensure that the distance metrics are meaningful, we apply the following preprocessing:

- **Normalization:** Numeric features are scaled using either Z-score or Min-Max normalization to prevent features with larger magnitudes from dominating the distance calculation.
- **Categorical Encoding:** Nominal features are handled with one-hot encoding. For specific cyclic features (e.g., month and day in the Forest Fires dataset), we implement a sine/cosine encoding to preserve the shape and relationship of temporal data.
- **Target Transformation:** For skewed regression targets, such as the burned area in the Forest Fires dataset, a $\ln(x + 1)$ transformation is applied.

2.2 Evaluation Protocol

To ensure statistical robustness, we employ a 5×2 cross-validation framework. The data is split into two halves; we train on half and test on the other, then reverse the roles. This process is repeated five times with different random splits.

- **Classification Metric:** Performance is measured using the Classification Error rate.
- **Regression Metric:** Performance is measured using Mean Squared Error (MSE).

3 Algorithms Implemented

This section describes the algorithms implemented for this project. We begin with the null model, which serves as a naive baseline against which all other methods are compared. We then describe the k-nearest neighbor classifier and regressor, including how distances are computed and how predictions are formed from a query point’s neighbors. Finally, we present the edited and condensed KNN algorithms used to reduce the training set, along with the hyperparameter tuning approach used to select k , γ , and ϵ for all methods.

3.1 Null Models

The null model serves as a baseline for comparison, as the predictions the null model makes are independent of the input feature values. For classification: The model identifies the most frequently occurring class label in the training set and then assigns that label to every instance in the test set. We then compute the classification error of these predictions. For regression: The model computes the average of the label column in the training set and assigns that value to every instance in the test set. We then compute the mean squared error of these predictions.

3.2 k-Nearest Neighbor Classification

The k-nearest neighbor (KNN) classifier predicts a label for a query instance by identifying the k training instances that are closest to it in feature space and having those neighbors determine the query's predicted class. For each query point, we compute its distance to every instance in the training set using Euclidean distance, sort the training instances by this distance, and select the k closest. The class label that occurs most frequently among the neighbors' labels becomes the predicted label. We treat k as a tuned hyperparameter (Section 3.6).

3.3 k-Nearest Neighbor Regression

The k-nearest neighbor regressor also predicts a label for a query instance by identifying the k training instances that are closest to it in feature space and having those neighbors determine the query's prediction. For each query point, we compute its distance to every instance in the training set using Euclidean distance, sort the training instances by this distance, and select the k closest. We then calculate the predicted value as a weighted average of the neighbors' label values, where the weight assigned to each neighbor is determined by a Gaussian kernel:

$$K(x, x_q) = \exp(-\gamma(x - x_q)^2) \quad (1)$$

where $(x - x_q)^2$ is the Euclidean distance between the query point x_q and a neighbor x , and γ is a bandwidth parameter controlling how quickly a neighbor's influence decays with distance. Closer neighbors receive exponentially larger weight than farther ones. The final prediction is the weighted average of the k neighbors' target values, normalized by the sum of their weights:

$$\hat{y}(x_q) = \frac{\sum_i K(x_q, x_i) y_i}{\sum_i K(x_q, x_i)} \quad (2)$$

for $i = 1, 2, \dots, k$. Small values of the bandwidth γ cause distant neighbors to contribute nearly as much as close ones, smoothing predictions. Large values of γ concentrate influence on the closest neighbors, making predictions more sensitive to local variation. We treat both k and γ as tuned hyperparameters (Section 3.6), selected together.

3.4 Edited KNN

Edited KNN reduces the training set by removing instances where their own nearest neighbor does not agree with them. For each training instance, we temporarily remove it from

the training set and classify it using its single nearest neighbor ($k = 1$) from the remaining instances. For classification, an instance is retained only if its 1-nearest neighbor prediction matches its true label and is discarded otherwise. For regression we retain an instance only if its 1-nearest neighbor prediction falls within an error threshold ϵ of its true value, and discard it otherwise. We treat ϵ as a tuned hyperparameter (Section 3.6). This procedure is applied once over the entire training set, evaluating each instance against the others before any points are removed. The result is a reduced set that ideally preserves the same predictions as the original data while discarding redundant or noisy instances.

3.5 Condensed KNN

Condensed KNN builds a minimal subset from scratch by keeping only the instances needed to correctly predict the rest of the data. We initialize the reduced set with a single random instance in the training set and treat all of the remaining instances as unprocessed. We then repeatedly pass over the remaining instances, and classify each one using its single nearest neighbor ($k = 1$) from the current reduced set. For classification, an instance is added to the reduced set if its label disagrees with the label predicted by its 1-nearest neighbor of the reduced set. For regression, an instance is added to the reduced set if its 1-nearest neighbor prediction from the reduced set falls outside an error threshold ϵ of its true value. We treat ϵ as a tuned hyperparameter (Section 3.6). We repeatedly pass over all of the instances not in the reduced set until no new instances are added to the reduced set. Instances that are correctly predicted by the reduced set at every pass are never added and are discarded as redundant. The result is a reduced set that correctly predicts all instances in the training set while omitting instances that are already well represented.

3.6 Hyperparameter Tuning

To tune the hyperparameters for these algorithms, we use random search. For a fixed number of iterations, we sample a candidate hyperparameter value (or combination of values), evaluate the resulting model, and retain whichever hyperparameter candidate achieves the lowest error. For k , we restrict sampling to odd integers in the range $[1, \sqrt{n}]$, where n is the number of instances in the training set. Restricting to odd values avoids ties in the plurality vote used by the KNN classification algorithm (Section 3.2). Bounding the upper end by $\sqrt{n}$ prevents k from growing unreasonably large relative to the training data. For γ , we sample on a logarithmic scale over the range $[0.0001, 10]$ as an ideal value may lie anywhere across several orders of magnitude, and a uniform draw would over sample large values. For ϵ , we sample uniformly from a range defined by 0.1% to 15% of the standard deviation of the regression target so the threshold scales with the spread of each dataset’s label.

For classification, we split the data into an 80/20 training/validation split, sample k candidate values, and select the k that minimizes classification error on the validation split. For regression, k and γ are tuned jointly. We split the data into an 80/20 training/validation split, sample k candidate values, calculate the prediction using γ , and select the (k, γ) pair that minimizes mean squared error on the validation split. For editing and condensing on regression tasks, ϵ is tuned separately. For each ϵ candidate, we construct the reduced training set using that error threshold and evaluate the resulting reduced set’s performance

on a validation split using KNN regression where $k = 1$. The ϵ value with the lowest mean squared error is selected.

4 Experimental Results

We evaluate the null model, k-nearest neighbor, Edited KNN, and Condensed KNN using 5×2 cross-validation on each of the six datasets, reporting classification error for the three classification datasets and mean squared error (MSE) for the three regression datasets. Hyperparameters were tuned via random search as described in Section 3.6. k was searched with 100 iterations for classification, a budget chosen because k is restricted to a small number of discrete odd values, so additional iterations beyond this point yield diminishing returns. k and γ were searched jointly with 250 iterations for regression, which reflects the added difficulty of the joint search. Since γ is continuous and spans several orders of magnitude, we require more samples to adequately cover the space. ϵ was searched with just 75 iterations for editing and condensing on regression tasks due to a substantially more expensive tuning algorithm. Every iteration of ϵ rebuilds the reduced training set from scratch, which is reflected in ϵ having fewer iterations. We confirmed after tuning that the selected hyperparameter values did not consistently fall near the boundaries of their respective search ranges. This indicates that the chosen iteration counts and ranges provided adequate coverage of the search space.

4.1 Classification Results

Table 1 reports classification error for the null model and the tuned unreduced, Edited, and Condensed KNN classifiers on the three classification datasets.

Dataset	Null Model	KNN (k)	Edited KNN (k)	Condensed KNN (k)
Breast Cancer	0.3448	0.0398 ($k = 13$)	0.0120 ($k = 1$)	0.2791 ($k = 5$)
Car Evaluation	0.2998	0.0716 ($k = 13$)	0.0459 ($k = 5$)	0.3404 ($k = 15$)
House Votes	0.3862	0.0869 ($k = 17$)	0.0290 ($k = 5$)	0.3701 ($k = 5$)

Table 1: Classification error (5×2 CV mean) for the null model, unreduced KNN, Edited KNN, and Condensed KNN across the three classification datasets. Tuned k shown in parentheses.

Unreduced and Edited KNN substantially outperform the null model on all three classification datasets, performing 76% to 97% better in each case. Condensed KNN’s performance relative to the null model is mixed. It improves on the null model for Breast Cancer and House Votes, but underperforms it on Car Evaluation.

4.2 Regression Results

Table 2 reports mean squared error for the null model and tuned unreduced, Edited, and Condensed regressors. Table 3 reports the tuned k , γ , and ϵ values for the unreduced, Edited, and Condensed regressors.

Dataset	Null Model	KNN	Edited KNN	Condensed KNN
Abalone	10.3928	4.9687	1.8282	5.3877
Computer Hardware	25742.7614	6163.6571	8477.8677	5033.5036
Forest Fires	1.9518	2.0536	0.1631	2.2688

Table 2: Mean squared error (5×2 CV mean) for the null model, unreduced KNN, Edited KNN, and Condensed KNN across the three regression datasets.

Dataset	KNN (k, γ)	Edited KNN (k, γ, ϵ)	Condensed KNN (k, γ, ϵ)
Abalone	23, 0.7481	19, 0.0003, 0.2926	25, 1.0899, 0.1326
Computer Hardware	13, 0.4328	5, 0.0834, 2.6104	9, 0.0002, 1.4577
Forest Fires	21, 2.2575	15, 1.8340, 0.0331	13, 0.0001, 0.0806

Table 3: Tuned hyperparameters for unreduced KNN, Edited KNN, and Condensed KNN on the regression datasets.

Edited KNN outperforms the null model on all three regression datasets. Condensed KNN improves on the null model for Abalone and Computer Hardware, but performs worse than the null model on Forest Fires. Unreduced KNN shows a wide range of performance relative to the null model. On Forest Fires it offers almost no improvement, while on Abalone and Computer Hardware it improves substantially. Notably, Computer Hardware is the only regression dataset on which unreduced KNN outperforms Edited KNN, indicating that editing did not uniformly help across all three datasets.

4.3 Training Set Reduction

Table 4 reports the size of the reduced training set produced by editing and condensing, expressed as a percentage of the original training set.

Dataset	Original Size	Edited KNN Retained	Condensed KNN Retained
Breast Cancer	699	95.9%	11.6%
Car Evaluation	1728	89.1%	26.6%
House Votes	435	93.6%	15.4%
Abalone	4177	19.3%	89.1%
Computer Hardware	209	12.4%	96.7%
Forest Fires	517	27.3%	84.3%

Table 4: Percentage of the original training set retained after editing and condensing, computed using each dataset’s tuned ϵ (regression) or default editing/condensing behavior (classification).

The two reduction methods behave in opposite ways depending on the task type. For the classification datasets, editing retains most of the training set, while condensing retains a much smaller subset. For the regression datasets, this pattern reverses. Editing retains

only a small fraction of the data, while condensing retains the large majority. We credit this reversal to the stricter numeric tolerance used for regression.

5 Analysis and Discussion

The experimental results provide strong evidence for the effectiveness of local proximity in predicting target values. Edited KNN outperformed the Null Model on all six datasets, strongly supporting our first hypothesis. However, unreduced KNN and Condensed KNN did not always outperform the Null Model. Unreduced KNN performed marginally worse than the Null Model on the Forest Fires dataset, and Condensed KNN performed marginally worse than the Null Model on the Forest Fires and Car Evaluation datasets. Our first hypothesis is therefore best supported when considering the tuned Edited KNN algorithm.

In classification tasks, Edited KNN proved vastly superior to Condensed KNN. For instance, in the House Votes dataset, Edited KNN achieved an error of 0.0290 compared to 0.3701 for Condensed KNN, which performed nearly as poorly as the Null Model. This suggests that for these specific datasets, removing noise from the boundaries (Editing) is more effective than attempting to build a minimal representative set (Condensing). This supports the accuracy component of our second hypothesis. However, Edited KNN did not achieve a substantial reduction across all tasks as our second hypothesis predicted. For classification, Edited KNN retained 89% to 96% of the training set rather than a much smaller subset.

Our third hypothesis, that Condensed KNN would achieve the highest rate of data reduction while maintaining acceptable performance, is supported only for the classification datasets. For classification, Condensed KNN retained the smallest fraction of the training set of any method (12% to 27%). For regression, however, this pattern reversed. Condensed KNN retained the majority of the training data (84% to 97%). This suggests that the mechanism driving Condensed KNN’s reduction on classification tasks does not transfer to regression. Our third hypothesis is therefore only partially supported, and appears to depend heavily on task type.

Regression results were more varied. While Edited KNN was highly effective for Forest Fires and Abalone, Condensed KNN outperformed it on the Computer Hardware dataset (MSE 5033.50 vs 8477.86). This indicates that the a-priori nature of the data (noise level vs. redundancy) dictates which reduction strategy is optimal. Furthermore, the wide variance in γ values—ranging from 0.0001 to 2.2575—confirms that kernel bandwidth tuning is critical for regression performance.

6 Conclusion

This project demonstrated the utility of nonparametric learning and the impact of data reduction on predictive accuracy. While standard KNN is a powerful baseline, Edited KNN generally provided the best balance of accuracy and noise reduction for classification. For regression, the Gaussian kernel allowed for a flexible approach to local averaging, though results were highly sensitive to the tuned bandwidth γ . Overall, the results highlight the necessity of preprocessing and hyperparameter tuning when dealing with diverse real-world datasets.

Appendix A. Main Lessons Learned

Through the implementation of this project, we learned the critical importance of feature scaling; without normalization, the Minkowski distance is skewed toward features with larger magnitudes. We also observed that the choice of k and γ can lead to significant overfitting or underfitting, reinforcing the need for a robust tuning pipeline like random search. Finally, we learned that "data reduction" is not a one-size-fits-all process. Removing noise (Editing) and removing redundancy (Condensing) serve different purposes and yield different results depending on the dataset.

Appendix B. Delineation of Work

- **Isaac Wilkey:** For the report, I wrote the Problem Statement and Hypothesis, the Experimental Approach and Project Design, and the Conclusion. I also contributed to the Algorithms Implemented section. Additionally, I implemented the core code for the Edited KNN and Condensed KNN data reduction algorithms.
- **Riley Smith:** For the report, I wrote the majority of the Algorithms Implemented section (Section 3), as well as the Experimental Results section (Section 4). For the programming side, I wrote the preprocessing, null, unreduced KNN, evaluation, and tuning algorithms. I also compiled the final results.